

RedrobRank Agentic Recruiter OS

Deterministic, CPU-only candidate discovery for the Redrob Data & AI Challenge

Team Name: Syntheon

Team Leader: Saurav Soni

Problem Statement: Intelligent Candidate Discovery

Hiring AI talent from 100K profiles is not a keyword

1. Keyword stuffing

Candidates can list AI buzzwords without real depth.

2. Ghost candidates

Strong profiles can be inactive, unavailable, or low-response.

3. Domain mismatches

Support, accounting, civil/mechanical, HR, and content roles may contain AI keywords.

4. Hallucinated explanations

Ranking systems fail trust if the reason does not match candidate facts.

RedrobRank was built to optimize relevance + trust + reproducibility.

RedrobRank turns 100K candidates into a ground

1. Extract evidence

Title, YOE, location, career, skills, Redrob behavioral signals.

2. Score fit

AI/retrieval, production, evaluation, systems, YOE, location, behavior.

3. Filter risk

Honeypots, keyword stuffers, non-fit roles, manager-only, low-signal.

4. Explain output

Every reason references actual candidate facts.

100K processed

100 final rows

CPU-only

No LLM/API

Validator passed

What the Senior AI Engineer JD actually needs

Must-have signals

- Retrieval / vector search / semantic search
- Ranking / recommender systems / personalization
- LLM / RAG / fine-tuning / NLP
- Production ML / MLOps / deployment
- Evaluation: NDCG, MRR, A/B testing
- Python + systems/data infrastructure

Negative signals

- Non-technical role mismatch
- Prompt-only or keyword-stuffed profiles
- Pure management without recent technical depth
- Pure research without production evidence
- Weak availability/engagement

Deterministic multi-signal ranking, not black-box

Components:

- TF-IDF lexical recall
- Structured AI skill score
- Skill trust: proficiency, duration, endorsements
- Career relevance with recency weighting
- Production / MLOps evidence
- Evaluation / experimentation evidence
- Redrob behavioral availability
- Location / relocation
- Risk penalties
- RRF fusion + deterministic tie-break

Sort = score descending, then candidate_id ascending.

Every recommendation is auditable

Main proof:

Reasoning must include actual:

- current title
- YOE
- location
- matched AI/retrieval terms
- production/eval evidence
- Redrob response/open/relocation signal

Validation cards:

- Official validator passed
- Internal validator passed
- Forensic audit passed
- 0 hallucination/rule failures
- 74 unique scores
- 100 unique reasonings

No LLM-generated justifications. Explanations are deterministic and evidence-grounded.

From raw candidate pool to validated submission

→ Candidate JSONL

→ Feature Extractor

→ TF-IDF + Structured Scorer

→ Grounded Reasoning

→ Validator

→ Stream Reader

→ Risk Detector

→ RRF Ranker

→ Top-100 CSV

→ Dashboard + Sandbox

Offline | CPU-only | no raw data committed | reproducible

Local-first architecture built for privacy and repro

CLI Ranker:

rank.py,
stream_candidates,
feature extraction,
scoring,
CSV writer,
reports.

Audit Layer:

validate_submission.py,
internal validation,
forensic_audit.py,
security scans.

Interfaces:

React/Vite dashboard
reads reports/*.json.

Streamlit sandbox runs
sample candidates locally.

No external APIs

no keys

no LLM calls

no backend upload of 100K data

Final output surfaces real AI/ML/retrieval profiles

Metrics

100,000 candidates processed | 100 ranked candidates | 74 unique scores | 100 unique reasonings
0 hallucination failures | CSV validator passed | Under 5 MB output

Top candidate preview table:

1. CAND_0018499 — Senior ML Engineer — 7.2 YOY — Noida
2. CAND_0005260 — Senior NLP Engineer — 5.2 YOY — Chennai
3. CAND_0046525 — Senior ML Engineer — 6.1 YOY — Pune
4. CAND_0081846 — Lead AI Engineer — 6.7 YOY — Jaipur
5. CAND_0042506 — Search Engineer — 4.2 YOY — Mumbai

Built like an evaluator-proof submission package

Tech:

- Python
- scikit-learn TF-IDF
- deterministic JSONL streaming
- React + Vite dashboard
- Streamlit sandbox

Quality:

- pytest / coverage
- ruff
- Bandit
- pip-audit
- npm audit
- forensic audit

Security:

- no raw dataset committed
- no LLM/API dependency
- no secrets
- local CPU execution
- sample sandbox only,
no full dataset upload

Submission package

Submission Assets

GitHub Repo: <https://github.com/Sauravssoni/Screeener>

Ranked CSV: `submissions/submission.csv`

Deck PDF: `docs/RedrobRank_Official_Submission_Deck.pdf`

Methodology PDF: `docs/RedrobRank_Methodology.pdf`

Audit: `reports/final_ranking_audit.md`

Sandbox: `streamlit run sandbox_app.py`

Dashboard: `npm run dev`

RedrobRank is not just a ranking script — it is a reproducible, auditable TalentOps eng